Calcolo Parallelo e Distribuito

openMP:

Una prova d'esame molto facile

Docente: Prof. L. Marcellino

Tutor: Prof. P. De Luca

INFORMAZIONI IMPORTANTI PER L'ESAME:

- **PROVA scritta: durata 2 ore**

Descrizione di un algoritmo parallelo relativo ad un problema assegnato, Calcolo Teorico di una o più metriche di valutazione (Speedup ed Efficienza, Isoefficienza, Ware-Amdhal), **decrizione e commento del codice relativo all'algoritmo**

- **PROVA orale (obbligatoria), ma solo se ammessi (scritto sufficiente)**

Esercitazione in aula

Si consideri il seguente problema:

$$m = \min_i(A\,x)$$

dove A è una matrice di dimensione $N \times M$, x è un vettore di dimensione M

Il metodo risulutivo deve eseguire le seguenti operazioni:

- calcolare il prodotto matrice-vettore $y = Ax$, utilizzando una strategia di parallelizzazione basata sulla suddivisione della matrice A per blocchi di colonne (II strategia);
- calcolare il valore minimo degli elementi del vettore y.

Assumendo l'utilizzo di np thread, determinare Speedup e Overhead, secondo la definizione classica, dell'intera procedura parallela, motivando tutti i passaggi.

... attenzione continua!

Considerato il seguente codice che risolve il problema illustrato:

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main() {

    int N = 1000, M = 1000;
    int i, j;

    double **A, *x, *y;

    A = (double **) malloc(N * sizeof(double *));
    for (i = 0; i < N; i++)
        A[i] = (double *) malloc(M * sizeof(double));

    x = (double *) malloc(M * sizeof(double));
    y = (double *) calloc(N, sizeof(double));

    for (i = 0; i < N; i++)
        for (j = 0; j < M; j++)
            A[i][j] = 1.0;

    for (j = 0; j < M; j++)
        x[j] = 1.0;

    #pragma omp parallel for reduction(+:y[:N])
    for (j = 0; j < M; j++)
        for (i = 0; i < N; i++)
            y[i] += A[i][j] * x[j];

    double m = y[0];

    #pragma omp parallel for reduction(min:m)
    for (i = 0; i < N; i++)
        if (y[i] < m)
            m = y[i];

    printf("Minimo = %f\n", m);

    return 0;
}
```

Domande varie di tipo tecnico…

Soffermiamoci sulla questione più importanti!

Il prodotto matXvet con la seconda strategia

Algoritmo con riduzione (distribuzione per colonne)

v1: la più semplice ma la meno performante

```
#pragma omp parallel for shared(B, a, result) private(i, j)
   for (j = 0; j < N; j++) {
      for (i = 0; i < N; i++) {
         #pragma omp atomic
         result[i] += B[i][j] * a[j];
      }
   }
```

Il secondo quesito

Algoritmo con riduzione (distribuzione per colonne)

v2: la prima strategia per la collezione dei vettori

```
#pragma omp parallel for shared(B, a, result) private(i, j)
  for (j = 0; j < N; j++) {
     double temp[N] = {0}; // risultato parziale per ogni colonna
     for (i = 0; i < N; i++) {
        temp[i] = B[i][j] * a[j];
     }
     #pragma omp critical
     {
        for (i = 0; i < N; i++) {
           result[i] += temp[i];
        }
     }
  }
```

Il secondo quesito

Algoritmo con riduzione (distribuzione per colonne)

v3: la seconda strategia per la collezione dei vettori

```
#pragma omp parallel for shared(B, a) private(i, j) reduction(+:result[:N])
  for (j = 0; j < N; j++) {
     for (i = 0; i < N; i++) {
        result[i] += B[i][j] * a[j];
     }
  }
```

result[:N] indica che la riduzione deve essere effettuata su un array result di lunghezza N

riduzione su array

disponibile a partire da OpenMP 4.5

VERIFICARE LA VOSTRA VERSIONE …